

Using C Utility Implementations to Verify Shell Scripts

A prototype study of compositional verification and LLM proof generation
Revised manuscript, 9 September 2026

Abstract

An AI assistant can generate a shell script and an explanation of what it does, but neither establishes that the script satisfies the user's request. Formal verification could check the script against a precise requirement, provided that the verifier has reliable definitions of the commands the script invokes. This study investigates using utilities' C implementations to supply those definitions, while specifying their library interactions separately. We develop a prototype around a byte-copying utility and a small shell fragment. In Lean, we connect an execution model of the utility's generated C representation to a shared-state interpreter, and prove that command and query compilation preserves the modeled behavior. The connection relies on trusted translation and modeling steps; it is not a proof of arbitrary Bash or host-system execution. Separate Coq/VST case studies examine reuse for GNU utility functions. A UTF-8 experiment ends without the intended C-conformance proof, identifying a limit of the verification approach. Finally, a small proof-generation experiment accepts 23 of 90 scored submissions under fixed checking rules. The result is a concrete, limited example of connecting utility behavior to script-level reasoning, rather than a complete system for verifying natural-language requests.

1. Research question

The motivating application is straightforward: a user asks an AI assistant to perform a task, the assistant generates a shell script, and a verifier checks that the script meets a formal statement of the task. For example, a request to copy input and then record success requires reasoning about both the copying command and the shell operator that controls whether the second command runs.

There are two distinct correctness questions. First, does the formal requirement capture what the user intended? Second, does the script satisfy that requirement? This project addresses machinery for the second question. It does not establish that an LLM translates natural-language intent correctly.

The central research question is:

Can we use a utility's C implementation as the basis for its behavior in a formal model, then reuse that behavior to verify scripts that compose the utility with other commands?

This approach seeks to avoid writing—or asking an LLM to invent—a separate behavioral specification for every utility. It still needs contracts for external operations such as reading and writing. Those contracts describe which results the environment may return and how those results affect memory and I/O.

Using an implementation as the specification has a deliberate consequence: a script proof can preserve the implementation's bugs. Establishing that a utility meets an independent standard is a different task. We explored that distinction with a separate UTF-8 calibration experiment.

We investigated three questions:

1. **Semantic connection:** can behavior derived from a utility implementation reach the formal model used for script verification?
2. **Composition and reuse:** can the model account for partial I/O and shared state across commands, and can library contracts be reused in other utility functions?
3. **Proof automation:** how often can language models regenerate selected proofs when the statements and checker are fixed?

The third question is a supporting experiment. It does not measure whether models generate correct shell scripts.

2. A running example

Our main utility, `relay`, reads bytes from standard input and writes them to standard output using a 32-byte buffer. A write may deliver only part of the buffer. A later write may fail. Consequently, proving that the utility “copies input” requires distinguishing bytes already delivered, bytes still pending, and input not yet read.

Consider two scripts in the supported fragment:

```
relay && mark
relay || mark
```

Here `mark` is a modeled command that appends a marker and returns success. It is not another verified C utility. In the first script, `mark` runs only if `relay` succeeds. In the second, it runs only if `relay` fails. A failure followed by a successful marker therefore makes the second script return success even though the copy failed. Checking the final exit status alone would miss that distinction.

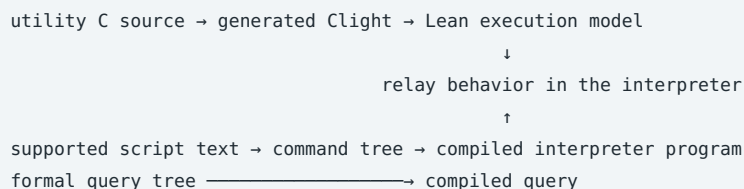
The model records remaining input, delivered bytes, undelivered bytes recorded by the model, future read and write outcomes, and read/write call counts. Commands share this state. A second command consumes the outcomes left by the first; it does not receive a reset environment.

We represent possible external I/O behavior using **schedules**: finite descriptions of the results returned by successive reads and writes. The proofs quantify over the permitted schedules, rather than checking only a collection of successful executions. Their applicability is nevertheless limited by the scheduled-I/O contracts and the resource assumptions described below.

3. Approach

3.1 From utility source to a script model

The prototype has two paths that meet in a Lean interpreter:



Clight is the C intermediate language produced by CompCert. A syntax-directed Python translator converts the generated relay representation into Lean data. We implement the subset of Clight operations used by this program in Lean, then prove that executing the generated representation agrees with a functional model of the relay. A further theorem connects that behavior to the relay command in the script interpreter.

The interpreter is the project's **state calculus**: a small formal language for state updates, action calls, conditions and control flow. Calling it a calculus does not mean that it implements all of Bash. The supported Lean command language contains relay, mark, sequencing, &&, || and parentheses.

This connection is stronger than testing two implementations on the same inputs: Lean checks a theorem relating their modeled executions. However, Lean does not verify the C-to-Clight frontend, the Python translator, or our transcription of Clight semantics against CompCert's own semantics. Those steps remain trusted. The theorem is about the resulting Lean execution model, not directly about a binary running on the host OS.

3.2 From scripts and queries to proofs

A tokenizer splits text into words and operators; a parser arranges those tokens into a command tree. For example, relay && mark becomes a conjunction whose two children are the relay and marker commands. The compiler translates that tree into the state calculus.

The supported text parser has a soundness theorem relative to a token grammar. We have not proved a separate character-level lexer theorem or equivalence to Bash's parser. Separately, the integration with the existing OCaml frontend includes checked identities for exported examples and empirical comparisons of executable translations; those comparisons are not proofs of the OCaml source.

A **query** is a formal Boolean property. The query language can inspect exit status, cumulative call counts, and lengths of the state lists, using comparisons and Boolean connectives. Queries are supplied as Lean data. There is no verified natural-language-to-query translator, and exact byte-sequence expressions are outside this query language.

The main compilation result can be stated informally as follows:

Given a supported command and query, a correctly represented initial state, the specified command bodies, and sufficient execution resources, the compiled program reaches the state described by the command model and computes the same Boolean answer as the query model.

There are two uses of this result. It ensures that compiling a query does not change what the query means. It also transfers a separately proved universal property of the command model to the compiled program. Compilation alone does not make an arbitrary query true; that property still needs a proof.

The resource assumptions have concrete purposes. **Fuel** bounds the interpreter's execution steps. **Integer headroom** ensures that accumulated counters remain representable. For a command with at most b relay calls and initial input length at most L , the budget reserves up to $b(L + 1)$ further reads and bL writes. The extra read accounts for EOF, including calls on empty input. These bounds are derived across the command structure, including short-circuit branches. The 32-byte limit is the relay's buffer size, not a 32-byte limit on the total input.

3.3 C verification and contract reuse

In parallel, we use Coq and the Verified Software Toolchain (VST) to prove properties of relay and selected GNU utility functions under explicit library contracts. These proofs address C memory and I/O reasoning and test which contracts can be reused.

The Coq and Lean developments are separate proofs. We do not import a Coq proof into Lean. The relay has the additional generated-source connection described above; the other C case studies do not become commands in the Lean script language.

4. Formal results and case studies

Question	Result	Scope
Does relay behavior reach the script model?	A Lean theorem connects the generated relay representation, its functional behavior, and the interpreter's relay command.	Reviewed Lean model of a Clight subset, scheduled I/O, sufficient fuel and counter capacity.
Does command composition preserve state?	Command and query compilation preserve the modeled outcome, including residual I/O schedules and cumulative counters.	The two-command Lean fragment described above.
Can other control-flow forms be related?	Raising and caught relay implementations have checked agreement with ordinary relay on return status and the seven tracked state fields.	Stated initial-state and resource premises.
Can I/O contracts be reused?	GNU utility body proofs and additional <code>head_bytes</code> and <code>wc_lines</code> body/wrapper proofs reuse parts of the I/O development.	Selected functions and wrappers, not whole GNU executables.
Can we prove an independent decoder specification?	The UTF-8 testing and specification work completed, but the intended C-conformance proof did not.	A negative calibration result.

4.1 What the script examples establish

One checked property says that successful relay && mark leaves nonempty delivered output. Relay alone does not satisfy that property: successful execution on empty input provides a counterexample. A second property says that relay || mark returns status zero, because the modeled marker succeeds when the relay fails.

These examples demonstrate the compilation and composition machinery. They are modest specifications: nonempty output does not identify a particular marker byte, and status zero does not establish that copying succeeded. The separate command theorem gives more precise byte effects than the compiled query language can express.

The relay connection accounts for partial writes and failures. Its C-side memory correspondence identifies the current pending buffer suffix. The accumulated record of earlier undelivered bytes is bookkeeping in the calculus, not a claim that those bytes remain in C memory.

A separate Coq shell development handles bounded pipes and redirections. Those operators are not supported by the Lean command compiler, and the two results should not be read as a single verified Bash implementation.

4.2 Additional GNU utility functions

The GNU development includes five body proofs and four linked verification components. The additional `head_bytes` and `wc_lines` cases test reuse beyond the purpose-built relay.

Both reuse the read contract. The write side exposes a limit of reuse: `head_bytes` needs a contract for buffered standard I/O rather than the existing unbuffered full-write contract. The verified `wc_lines` function uses reading, error handling, byte search and output pointers; it is not a write-routine case study. These results show reuse within selected C functions, with adaptation at different library interfaces.

4.3 Why UTF-8 was included

The shell application mixes two difficulties: constructing trustworthy C verification machinery and choosing a useful specification for command behavior. UTF-8 supplies a separate, independently defined correctness target. We used it to investigate the verification machinery without making it the main research objective.

The experiment checked a Unicode 16 table specification and compared 1,190,417 finite cases against a fixed C decoder, with additional directed checks and sanitizers. It reached its recorded stopping condition without a VST proof that the C decoder implements the specification or a proof about its caller. The testing is evidence about the tested executions; it does not replace the missing conformance proof.

5. LLM proof-generation experiment

5.1 Design

We selected three previously proved Lean theorems about nested-state framing and preservation. For each theorem, we varied the available helper lemmas and asked three model/harness combinations for five fresh-context attempts per condition: 90 scheduled calls in total. First-pass attempts received no iterative compiler feedback.

The checker froze theorem statements and structural definitions, rejected prohibited shortcuts, compiled each submission, inspected its assumptions, and compared its theorem type with the expected type. A generated proof could not count as successful by weakening the theorem or altering the task.

An earlier 12-call diagnostic is reported separately. Its helper-removal experiment was ineffective because the relevant contract remained available in both conditions. We do not pool it with the expanded study.

5.2 Results

Recorded model and harness	Scored attempts	Accepted proofs
Claude Fable / Claude CLI	30	18
Claude Sonnet / Claude CLI	30	2
Grok 4.6 / Pi	30	3
Total	90	23

Three original calls returned responses but encountered stale checker-directory collisions. Counting those calls as failures gives 22 accepted proofs out of the original 90. Replacing them with the three recorded makeup calls gives the table's 23/90. There were therefore 93 first-pass calls, of which 90 form the replacement-based scored set. That set also contains 61 build failures and six timeout or missing-code outcomes.

All seven accepted submissions in the helper-deleted condition came from the Fable combination. Removing those helpers was not a hard barrier to finding a proof. A separate assisted experiment used six calls across three attempt records, exceeding its planned three calls; none was accepted. A further three-run agent-assisted collaboration experiment accepted one proof. It was not a human baseline.

5.3 Interpretation

These observations measure proof regeneration on a small set of tasks from this project. They do not measure script-generation accuracy or show that models reliably formalize user intent. Model and harness vary together, so the table cannot isolate the effect of the model from the effect of its tooling. Five attempts per experimental cell and the availability of alternative proofs also limit conclusions about helper reuse.

The clearest methodological result is that fixed statements and independent checking provide a concrete acceptance criterion. The observed success rates remain specific to the recorded tasks and configurations.

6. Relation to prior work and limitations

The project's [prior-art review](#) examines existing work on compositional I/O verification, C-to-interaction-tree refinement, executable effect models and library specifications. In particular, the work of Penninckx and colleagues on modular I/O verification and Koh and colleagues on a verified network server already demonstrates central ideas in specifying I/O and connecting C behavior to higher-level models. This study does not claim to introduce those ideas.

The intended contribution is their use in a concrete path from utility behavior to script/query reasoning, together with evidence about contract reuse and proof-generation checking. Establishing a publishable novelty claim requires a sharper comparison with that prior work. The present implementation and experiments alone do not establish one.

The main limitations are substantive:

- **Intent is not verified.** A proof that a script satisfies a query does not prove that the query captures the user's request.
- **The script language is small.** The Lean result covers relay, a modeled marker and selected control operators. It does not cover arbitrary Bash or whole GNU programs.
- **The source connection contains trusted steps.** CompCert translation, the Python dump and the reviewed Lean Clight semantics are not connected by a complete translation-correctness proof. The memory model uses total byte values and does not establish CompCert's uninitialized-memory behavior.
- **External behavior is assumed through contracts.** The theorems concern permitted scheduled reads and writes, not all behavior of a real OS or libc implementation.
- **Frontend evidence has different strengths.** Token-grammar soundness and selected exported identities are checked, while character-level lexing and OCaml-source correspondence remain outside the proof.
- **C and Lean results have distinct coverage.** Only relay has the source connection into the Lean script fragment. The Coq results for other utilities and shell operators are not imported into Lean.

These limitations locate the remaining work toward the motivating application. Completion of this prototype and its research record should not be confused with completion of that application.

7. Artifact and evidence guide

The source archive was built and independently checked after the original manuscript was written. Its 2,465 payload files matched their recorded hashes. The main fresh replay contains 30 entries: 29 passed, and one experimental host-OCaml check was skipped while its pinned-container counterpart passed. Two separately replayed supplements cover the exported-marker connector and the generated-source connection; they are not entries in that 30-item set.

The September 9 revision changes the exposition, not the proofs or experimental results. The frozen delivery retains the earlier manuscript, and the [previous text](#) is also preserved here. Toolchain requirements, checksums and final delivery records are in the [delivery receipt](#).

Evidence	Where to read it
Source-to-script connection and its assumptions	Source connection acceptance , Lean theorem source
Command and query compilation	Command proofs , query proofs
Independent UTF-8 calibration	Calibration results
Proof-generation design and accounting	Protocol , corrected report , results
Original requirements and detailed proof record	Requirements , proof chain
Literature and reuse analysis	Prior-art review

8. Conclusion

We built a small example of using utility implementation behavior in script-level formal reasoning. The relay connection and command/query theorems establish that the chosen modeled behavior can be composed and checked across shared state, subject to explicit translation, environment and resource assumptions. The additional C functions show both opportunities and limits for library-contract reuse. The UTF-8 result and proof-generation experiment identify areas where the current machinery falls short.

The next scientific question is how far this connection can be extended while reducing trusted translation and modeling steps. The longer-term application also requires a defensible way to connect user intent to the formal query. Neither question is settled by the current prototype.